Q.1 (a)

Define the following terms.

1. Metadata: Data about data. It describes the structure, constraints, and type of data stored in the database. (e.g., the name of a table, the data type of a column, etc.).

2. Schema: The logical structure of the entire database. It defines the tables, attributes, data types, and constraints.

3. Data Dictionary: A centralized repository of information about the data, essentially storing metadata. It contains details about schema, constraints, users, and other database information.

Q.1 (b)

Write down advantages of Database Management system.

· Reduces Data Redundancy: Minimizes duplication of data.

· Data Sharing: Allows data to be shared by multiple users and applications.

· Data Integrity: Maintains accuracy and consistency of data using constraints.

· Data Security: Provides controlled access to data via authorization.

· Backup and Recovery: Protects data from loss due to system failures.

· Data Independence: Allows changes in the schema without affecting application programs.

Q.1 (c)

Explain Responsibilities of DBA.

A Database Administrator(DBA) is responsible for:

· Schema Definition: Defining the database schema.

· Storage Structure and Access Method Definition: Deciding how data is stored and accessed.

· Granting of Authorization: Providing permissions to users for data access and modification.

· Routine Maintenance: Performing backups, recovery, and tuning database performance.

OR Q.1 (c)

What is data abstraction? Explain three level ANSI SPARC architecture in detail.

Data Abstractionis the process of hiding the complexity of the database system from end-users, providing them with a simplified view.

Three-Level ANSI-SPARC Architecture:

1. Internal Level (Physical Level): Describes how the data is physically stored in the database. It deals with storage structures, file organization, and access paths.

2. Conceptual Level (Logical Level): Describes what data is stored and the relationships among them. It is a community view of the database, hiding physical storage details.

3. External Level (View Level): Describes the part of the database that a particular user group is interested in. It hides the rest of the database from that user.

Q.1 (a)

Explain three-level database architecture.

(Same as OR Q.1 (c) above)

Q.1 (b)

Explain Total Participation and Partial Participation with example.

· Total Participation: Every entity in the entity set must participate in the relationship. It is represented by a double line in the ER diagram.

· Example: The relationship Enrolls between Student and Course is total if every student must enroll in at least one course.

· Partial Participation: Not every entity in the entity set is forced to participate in the relationship. It is represented by a single line.

· Example: The relationship Manages between Employee and Department is partial if not every employee manages a department.

Q.1 (c)

Explain advantages of DBMS over file management systems.

· Reduces Data Redundancy: DBMS controls redundancy, while file systems often have duplicate data across multiple files.

· Data Consistency: DBMS ensures consistent data, unlike file systems where updates in one file may not reflect in another.

· Data Sharing: DBMS allows concurrent data sharing, which is complex in file systems.

· Data Integrity: DBMS enforces integrity constraints (e.g., primary key, foreign key).

· Security: DBMS provides better security mechanisms.

· Backup and Recovery: DBMS has built-in backup and recovery features, which are manual and difficult in file systems.

OR Q.1 (c)

List out various data models. Explain any two in brief.

Various Data Models:Hierarchical, Network, Relational, Entity-Relationship, Object-Oriented.

· Relational Model: Data is organized into tables (relations) of rows and columns. It is the most widely used model. (e.g., Oracle, MySQL).

· Entity-Relationship Model: It is a conceptual model based on real-world entities and their relationships. It is represented using ER diagrams.

Q.2 (a)

Differentiate Schema vs Instance

Basis Schema Instance

Definition The overall logical design of the database. The actual data stored in the database at a particular time.

Change Frequency Changes infrequently. Changes frequently with every insert, update, delete.

Analogy Blueprint of a building. The actual building with people and furniture.

Q.2 (b)

Explain Specialization with example.

Specialization is a top-down process where an entity is divided into sub-entities based on their distinguishing characteristics.

· Example: The entity Person can be specialized into Employee and Student. An Employee has attributes like salary, and a Student has attributes like CGPA.

Q.2 (c)

What is ER diagram? Explain different symbols used in E-R diagram with example.

AnER Diagram is a visual representation of the logical structure of a database using Entity-Relationship models.

Symbols:

· Rectangle: Represents an Entity Set. (e.g., Student, Course)

· Ellipse: Represents an Attribute. (e.g., stu_id, course_name)

· Diamond: Represents a Relationship Set. (e.g., Enrolls)

· Line: Connects attributes to entities and entities to relationships.

· Double Rectangle: Represents a Weak Entity. (e.g., Dependent weak on Employee)

· Double Line: Represents Total Participation.

· Double Ellipse: Represents a Multivalued Attribute. (e.g., phone_number)

OR Q.2 (a)

Differentiate DA vs DBA.

Basis Data Administrator (DA) Database Administrator (DBA)

Focus Strategic, business-oriented. Technical, implementation-oriented.

Responsibility Defines data policies, standards, and planning. Implements the database as per the DA's design.

Role Managerial role. Technical role.

OR Q.2 (b)

Explain Generalization with example.

Generalization is a bottom-up process where multiple entity sets are synthesized into a higher-level,more generic entity set based on common features.

· Example: The entities Car and Truck can be generalized into a superclass Vehicle with common attributes like vehicle_id and model.

OR Q.2 (c)

What is attribute? Explain different types of attributes with example.

AnAttribute is a property or characteristic of an entity or relationship.

Types:

· Simple Attribute: Cannot be divided further. (e.g., stu_id)

· Composite Attribute: Can be divided into smaller parts. (e.g., name -> first_name, last_name)

· Single-Valued Attribute: Holds a single value. (e.g., age)

· Multi-Valued Attribute: Holds multiple values. (e.g., phone_number)

· Derived Attribute: Value can be derived from another attribute. (e.g., age can be derived from date_of_birth)

· Key Attribute: Uniquely identifies an entity. (e.g., stu_id)

Q.2 (a)

Explain Mapping Cardinalities.

Mapping Cardinalities define the number of entities to which another entity can be associated via a relationship set.

Types:

1. One-to-One (1:1): One entity in A is associated with at most one entity in B, and vice versa. (e.g., Husband - Wife)

2. One-to-Many (1:N): One entity in A can be associated with many entities in B, but an entity in B is associated with at most one entity in A. (e.g., Mother - Child)

3. Many-to-One (N:1): Inverse of One-to-Many. (e.g., Student - College)

4. Many-to-Many (M:N): One entity in A associated with many in B, and one in B associated with many in A. (e.g., Student - Course)

Q.2 (b)

Explain Outer Join operation in Relational Algebra.

An Outer Join is an extension of the join operation that keeps all tuples from one or both relations,even if they have no matching tuple in the other relation.

· Left Outer Join (⟕): Keeps all tuples from the left relation, padding with nulls for missing attributes from the right.

· Right Outer Join (⟖): Keeps all tuples from the right relation.

· Full Outer Join (⟗): Keeps all tuples from both relations.

Q.2 (c)

Explain concept of Specialization and Generalization with example.

(Combination of Q.2(b) and OR Q.2(b) above)

· Specialization (Top-down): Creating sub-classes from a superclass. Person -> Employee, Student.

· Generalization (Bottom-up): Creating a superclass from sub-classes. Car, Truck -> Vehicle.

OR Q.2 (a)

Explain different types of Keys in Relational Algebra.

· Super Key: A set of one or more attributes that can uniquely identify a tuple.

· Candidate Key: A minimal super key (no subset is a super key).

· Primary Key: The chosen Candidate Key to uniquely identify tuples in a relation.

· Foreign Key: An attribute in one table that is the Primary Key in another table, used to link two tables.

OR Q.2 (b)

Explain types of attributes in ER-diagram with suitable example.

(Same as OR Q.2(c) above)

OR Q.2 (c)

Explain SELECT, PROJECT, UNION and SET-INTERSECTION operation with suitable example.

Consider relations R(A,B) and S(A, B).

· SELECT (σ): Selects tuples that satisfy a given predicate.

· σ_(B="IT")(Student) → Returns all students from the IT department.

· PROJECT (π): Selects specific columns (attributes).

· π_(name, city)(Student) → Returns only the name and city of all students.

· UNION (∪): Returns all distinct tuples present in either relation.

· R ∪ S → Tuples in R or S or both.

· SET INTERSECTION (∩): Returns all distinct tuples present in both relations.

· R ∩ S → Tuples that are in both R and S.

Q.3 (a)

Explain the GRANT and REVOKE statement in SQL.

These are DCL(Data Control Language) commands used for user permissions.

· GRANT: Gives user privileges to the database.

· GRANT SELECT, INSERT ON Employee TO user1;

· REVOKE: Takes back privileges granted from the user.

· REVOKE INSERT ON Employee FROM user1;

Q.3 (b)

Explain following Character functions.

1. INITCAP: Converts the first letter of each word to uppercase and the rest to lowercase.
   · SELECT INITCAP('hello world') FROM DUAL; → 'Hello World'
2. SUBSTR: Returns a substring from a string, given a start position and length.
   · SELECT SUBSTR('Database', 1, 4) FROM DUAL; → 'Data'

Q.3 (c)

Consider following tables and write answers for the given queries.

1. SELECT * FROM stud_master WHERE dept = 'IT';
2. SELECT name FROM stud_master WHERE name LIKE 'p%';
3. INSERT INTO stud_master VALUES (101, 'John', 'New York', 'IT');
4. ALTER TABLE stud_master ADD gender VARCHAR(10);
5. SELECT COUNT(*) FROM stud_master;
6. SELECT * FROM stud_master ORDER BY enroll_no DESC;
7. DROP TABLE stud_master;

OR Q.3 (a)

Explain equi join with example in SQL.

An Equi Join is a type of inner join that combines tables based on the equality condition of a common column.

· Example:

```sql
SELECT e.name, d.dept_name
FROM Employee e, Department d
WHERE e.dept_id = d.dept_id;
```

OR Q.3 (b)

Explain following Aggregate functions.

1. MAX: Returns the maximum value in a column.
   · SELECT MAX(salary) FROM Employee;
2. SUM: Returns the sum of values in a numeric column.
   · SELECT SUM(quantity) FROM Orders;

OR Q.3 (c)

Write SQL queries for the following table:

1. CREATE TABLE PRODUCT_Master (prod_no NUMBER PRIMARY KEY, prod_name VARCHAR(20), profit NUMBER, quantity NUMBER, sell_price NUMBER, cost_price NUMBER);

2. INSERT INTO PRODUCT_Master VALUES (1, 'Laptop', 25000, 10, 75000, 50000);

3. SELECT * FROM PRODUCT_Master WHERE profit > 20000;

4. DELETE FROM PRODUCT_Master WHERE quantity < 5;

5. UPDATE PRODUCT_Master SET profit = profit * 1.02 WHERE sell_price > 5000;

6. ALTER TABLE PRODUCT_Master ADD total_price NUMBER;

7. SELECT DISTINCT prod_name FROM PRODUCT_Master;

Q.3 (a)

Differentiate Primary Key and Foreign Key constraint.

Basis Primary Key Foreign Key

Purpose Uniquely identifies a tuple in its own table. Creates a link between two tables.

Null Values Cannot accept NULL values. Can accept NULL values.

Duplication Cannot have duplicate values. Can have duplicate values.

Q.3 (b)

Explain DUAL table and SYSDATE with example.

· DUAL Table: A special one-row, one-column table present by default in Oracle databases. It is useful for performing calculations or calling functions.

· SELECT 15+10 FROM DUAL; → Output: 25

· SYSDATE: A function that returns the current date and time of the database server.

· SELECT SYSDATE FROM DUAL; → Output: Current Date/Time

Q.3 (c)

Write SQL queries to use various numeric functions.

```sql
SELECT ABS(-15) AS Absolute, -- 15
    CEIL(15.2) AS Ceiling, -- 16
    FLOOR(15.9) AS Floor, -- 15
    SQRT(25) AS SquareRoot, -- 5
    POWER(12, 6) AS Power, -- 2985984
    MOD(10, 3) AS Modulus, -- 1
    SIGN(-5) AS Sign -- -1
FROM DUAL;
```

OR Q.3 (a)

Explain Unique and Check Constraint with suitable example.

· UNIQUE Constraint: Ensures all values in a column are different.

· CREATE TABLE Student (stu_id NUMBER UNIQUE, name VARCHAR(20));

· CHECK Constraint: Ensures that all values in a column satisfy a specific condition.

· CREATE TABLE Employee (age NUMBER CHECK (age >= 18));

OR Q.3 (b)

Explain structure of PL/SQL block.

A PL/SQL block has three sections:

1. DECLARE (Optional): For declaring variables, constants, cursors.
2. BEGIN (Mandatory): Contains the executable statements.
3. EXCEPTION (Optional): For handling runtime errors.
4. END (Mandatory): Marks the end of the block.

```plsql
DECLARE
   message VARCHAR2(20) := 'Hello, World!';
BEGIN
   DBMS_OUTPUT.PUT_LINE(message);
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error occurred');
END;
```

OR Q.3 (c)

Consider the following table and solve queries:

I)CREATE TABLE BRANCH (branchid NUMBER PRIMARY KEY, branchname VARCHAR(20), address VARCHAR(50));

II)CREATE TABLE EMPLOYEE (empid NUMBER PRIMARY KEY, name VARCHAR(20), post VARCHAR(20), gender CHAR(1), birthdate DATE, salary NUMBER, branchid NUMBER, FOREIGN KEY (branchid) REFERENCES BRANCH(branchid));

III)SELECT * FROM EMPLOYEE e, BRANCH b WHERE e.branchid = b.branchid AND b.branchname = 'Ahmedabad';

IV)SELECT * FROM EMPLOYEE WHERE EXTRACT(YEAR FROM birthdate) = 1998;

V)SELECT * FROM EMPLOYEE WHERE gender = 'F' AND salary > 5000;

VI)SELECT b.address FROM BRANCH b, EMPLOYEE e WHERE e.branchid = b.branchid AND e.name = 'Ajay';

Q.4 (a)

Explain fully functional dependency with example.

A functional dependency X→ Y is a full functional dependency if removing any attribute A from X means that the dependency no longer holds.

· Example: In Student_Project(StuID, ProjID, StuName, ProjName, Hours), {StuID, ProjID} → Hours is a full FD because neither StuID nor ProjID alone can determine Hours.

Q.4 (b)

Relational Algebra Expressions for Employee table queries.

(Examples based on an Employee(emp_id, name, dept, salary) table)

· List all employees: π_(emp_id, name, dept, salary)(Employee)

· Find employees in 'IT' dept: σ_(dept='IT')(Employee)

· Find names and salaries: π_(name, salary)(Employee)

Q.4 (c)

What are the criteria of 2NF? Normalize given table into 2NF.

Criteria for 2NF:

1. The table must be in 1NF.

2. It should not have Partial Dependency (no non-prime attribute should be dependent on a proper subset of the candidate key).

Normalization Example:

Table:Order_Details(OrderID, ProductID, ProductName, Quantity)

· Partial Dependency: ProductID → ProductName (ProductName depends on only part of the PK).

· Solution: Decompose into:

  · Order_Items(OrderID, ProductID, Quantity) [PK: (OrderID, ProductID)]

  · Products(ProductID, ProductName) [PK: ProductID]

OR Q.4 (a)

Explain 3NF with example.

A table is in3NF if:

1. It is in 2NF.

2. It has no transitive dependency (no non-prime attribute depends on another non-prime attribute).

· Example: Employee(emp_id, dept, dept_head)

· Here, emp_id → dept and dept → dept_head. So, dept_head is transitively dependent on emp_id.

· Solution: Decompose into Employee(emp_id, dept) and Department(dept, dept_head).

OR Q.4 (b)

Relational Algebra questions for Students (Name, SPI, DOB, Enrollment No)

(Examples based on Student(EnrollmentNo, Name, SPI, DOB) table)

· List all students: π_(EnrollmentNo, Name, SPI, DOB)(Student)

· Find students with SPI > 9: σ_(SPI>9)(Student)

· Find names and DOB: π_(Name, DOB)(Student)

OR Q.4 (c)

What are the criteria of 1NF? Normalize given table into 1NF with two different techniques.

Criteria for 1NF:A relation is in 1NF if every attribute contains only atomic (indivisible) values.

Normalization Techniques:

Table:Student(StuID, Name, {Course}) where Course is multi-valued.

1. Flattening the Table (Repeating Groups): Create multiple rows for each course.

· Student_Course(StuID, Name, Course)

2. Creating a Separate Table: Remove the multi-valued attribute to a new table linked by the primary key.

· Student(StuID, Name)

· Student_Courses(StuID, Course)

Q.4 (a)

Explain Referential Integrity with suitable example.

Referential Integrity is a constraint that ensures a foreign key value in one table must always match a primary key value in the referenced table or be NULL.

· Example: In Employee(dept_id) and Department(dept_id), dept_id in Employee is a FK to Department. You cannot have a dept_id in Employee that does not exist in Department.

Q.4 (b)

Differentiate Partial and Full Functional Dependency.

Basis Partial Functional Dependency Full Functional Dependency

Definition A non-prime attribute depends on a part of the candidate key. A non-prime attribute depends on the entire candidate key.

Eliminated in 2NF Already addressed in 2NF.

Q.4 (c)

Explain 3rd Normal Form with example.

(Same as OR Q.4(a) above)

OR Q.4 (a)

Explain Importance of Normalization.

· Reduces Data Redundancy: Eliminates duplicate data.

· Avoids Data Anomalies: Prevents update, insertion, and deletion anomalies.

· Improves Data Integrity: Ensures accuracy and consistency of data.

· Simplifies Database Design: Makes the design more logical and efficient.

OR Q.4 (b)

Differentiate Prime Attributes and Non-Prime Attributes.

Basis Prime Attributes Non-Prime Attributes

Definition Attributes that are part of any candidate key. Attributes that are not part of any candidate key.

Example In Student(StuID, SSN, Name), StuID and SSN are prime. In the same table, Name is non-prime.

OR Q.4 (c)

Explain 2nd Normal Form with example.

(Same as Q.4(c) above)

Q.5 (a)

Explain ACID properties of transaction.

· Atomicity: A transaction is treated as a single, indivisible unit. It either executes completely or not at all.

· Consistency: A transaction must take the database from one consistent state to another.

· Isolation: The execution of one transaction should not be interfered with by other concurrent transactions.

· Durability: Once a transaction is committed, its changes are permanent and survive system failures.

Q.5 (b)

Create STUDENT table with constraints:

```sql
CREATE TABLE STUDENT (
   stu_id NUMBER PRIMARY KEY,
   stu_name VARCHAR(50) NOT NULL,
   Branch VARCHAR(20) CHECK (Branch IN ('IT', 'Computer', 'Electrical', 'Civil'))
);
```

Q.5 (c)

What is trigger? Write syntax & create trigger before inserting record.

ATrigger is a stored PL/SQL program unit that is automatically executed in response to a specific event (DML) on a table.

Syntax:

```plsql
CREATE [OR REPLACE] TRIGGER trigger_name
BEFORE | AFTER | INSTEAD OF
INSERT | UPDATE | DELETE [OF column_name]
ON table_name
[FOR EACH ROW]
[DECLARE]
BEGIN
   -- trigger body
END;
```

Example:

```plsql
CREATE OR REPLACE TRIGGER before_insert_student
BEFORE INSERT ON STUDENT
```

```
FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE('A new student is about to be inserted.');
END;
```

OR Q.5 (a)

Explain problems of concurrency control in transaction.

· Lost Update: Two transactions update the same data item, and one update overwrites the other.

· Dirty Read: A transaction reads data written by an uncommitted transaction.

· Non-Repeatable Read: A transaction reads the same row twice and gets different values because another transaction modified it in between.

· Phantom Read: A transaction re-executes a query and finds a set of rows that has changed due to another committed transaction.

OR Q.5 (b)

Create STUDENT table with constraints:

```sql
CREATE TABLE STUDENT (
  stu_id VARCHAR(10) PRIMARY KEY CHECK (stu_id LIKE 'S%'),
  stu_name VARCHAR(50),
  Branch VARCHAR(20)
```

);

```

OR Q.5 (c)

What is Explicit cursor? Explain with example.

AnExplicit Cursor is a programmer-defined cursor for more precise control over context area. It is defined in the declaration section of PL/SQL.

Example:

```plsql
DECLARE
  CURSOR c_student IS SELECT stu_id, name FROM STUDENT;
  v_id STUDENT.stu_id%TYPE;
  v_name STUDENT.name%TYPE;
BEGIN
  OPEN c_student;
  LOOP
    FETCH c_student INTO v_id, v_name;
    EXIT WHEN c_student%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name);
  END LOOP;
  CLOSE c_student;
END;
```
```

Q.5 (a)

Explain Transaction states with proper diagram.

(Diagram Description)A transaction goes through the following states:

1. Active: Initial state where the transaction is executing.

2. Partially Committed: After the final statement is executed.

3. Failed: When normal execution can no longer proceed.

4. Aborted: After the transaction is rolled back and the database is restored to its state before the transaction started.

5. Committed: After the transaction completes successfully and its changes are saved permanently.

Q.5 (b)

Explain any two DDL commands with suitable example.

· CREATE: Used to create a new database object.

· CREATE TABLE Employee (id NUMBER, name VARCHAR(30));

· ALTER: Used to modify the structure of an existing database object.

· ALTER TABLE Employee ADD (salary NUMBER);

Q.5 (c)

Explain ACID Properties in detail.

(Same as Q.5(a) above)

OR Q.5 (a)

What is two phase locking technique?

Two-Phase Locking(2PL) is a concurrency control protocol where a transaction cannot acquire any new locks after it has released a lock. It has two phases:

1. Growing Phase: The transaction can acquire locks but cannot release any.

2. Shrinking Phase: The transaction can release locks but cannot acquire any new ones.

This ensures serializability.

OR Q.5 (b)

Explain any two DML commands with suitable example.

· INSERT: Adds new rows to a table.

· INSERT INTO Employee VALUES (1, 'Alice', 50000);

· UPDATE: Modifies existing rows in a table.

· UPDATE Employee SET salary = 55000 WHERE id = 1;

OR Q.5 (c)

List problems of concurrency control and explain any two in detail.

(List from OR Q.5(a))

· Lost Update: When two transactions T1 and T2 read the same data and then update it, T2's update overwrites T1's update, which is lost.

· Dirty Read: A transaction T1 reads data that has been updated by another transaction T2 which has not yet been committed. If T2 rolls back, T1 has read data that never officially existed.